

Fatmin Labs: K3s Single-Node Lab

Project Goals & Architecture Plan

Project Context: The objective of this project is to architect and deploy a foundational Kubernetes learning environment using K3s on a single-node Red Hat Enterprise Linux (RHEL) virtual machine. This lab validates core Kubernetes API primitives while implementing enterprise-grade storage architectures, explicitly bridging local block storage (LVM) and external hypervisor-backed network storage (NFS).

1. Infrastructure Specifications

The baseline virtual machine requires the following configuration to ensure adequate overhead for control plane components and deployed workloads:

Resource	Allocation	Notes
Operating System	RHEL	Base environment supporting necessary storage tooling (LVM, nfs-utils).
Compute	2 vCPUs	Minimum for combined control-plane and worker-node operations.
Memory	4 GB - 8 GB RAM	Scale toward 8 GB to support future telemetry and monitoring stacks.
Root Disk	20 GB - 40 GB	Primary virtual disk for the OS and base K3s binaries.

Resource	Allocation	Notes
Secondary Disk (Storage Track A)	TBD (Block)	Raw virtual disk assigned for LVM creation and local volume provisioning.

2. Architectural Core Components

- **Distribution:** K3s (containerd runtime, embedded SQLite datastore)
- **Networking:** Flannel (CNI)
- **Ingress:** Traefik Controller
- **Storage Paradigms:**
 - ReadWriteOnce (RWO): K3s Local Path Provisioner backing into a dedicated Logical Volume (LVM) with an XFS filesystem.
 - ReadWriteMany (RWX): `nfs-subdir-external-provisioner` linked to a hypervisor-exported NFS share.

3. Project Execution Checklist

Phase 1: Workload Primitives

- Deploy a simple standalone Nginx Pod to validate ephemeral behavior.
- Wrap the Pod in a Deployment manifest, scale replicas to 3, and test self-healing by manually deleting a Pod.
- Establish a custom namespace (e.g., lab-apps) and target deployments to it for logical isolation.

Phase 2: Networking & Exposure

- **ClusterIP:** Expose a Deployment internally and validate internal cluster DNS via a busybox container.
- **NodePort:** Expose a workload dynamically to the VM's external IP address.
- **Ingress:** Map a hostname/URL path through the Traefik Ingress controller to route external traffic to internal services.

Phase 3: Persistent Storage (Dual Architecture)

Track A: Local Block Storage (RWO)

- Provision secondary virtual disk from the hypervisor and attach to RHEL node.
- Construct LVM configuration (Volume Group, Logical Volume) formatted with XFS.
- Mount the logical volume and reconfigure the K3s Local Path Provisioner to target the mount point.
- Deploy a PostgreSQL Pod with an RWO PersistentVolumeClaim (PVC), write test data, and cycle the Pod to verify persistence.

Track B: NFS External Provisioning (RWX)

- Export an NFS directory from the hypervisor.
- Validate nfs-utils and rpcbind functionality on the RHEL guest OS.
- Deploy the nfs-subdir-external-provisioner into the cluster.
- Deploy an Nginx Deployment scaled to 3 replicas, mounting a single RWX PVC to all replicas simultaneously to serve a shared index.html.

Phase 4: Configuration Management

- Utilize **ConfigMaps** to mount non-sensitive configuration files or environment variables into test Pods.
- Utilize **Secrets** to securely inject mock database credentials as environment variables.

Phase 5: Operational Troubleshooting

- Extract application behavior using kubectl logs -f.
- Diagnose live container states by dropping into an interactive shell via kubectl exec -it.
- Bypass Services entirely by securing local administrative tunnels using kubectl port-forward.

Document generated for project tracking and deployment validation.